ColdTrace Operations App: build brief for Newly

Nov 11, 2026 · @Muedi

Build a professional operations app that tells a cold-chain operations lead which storage unit is failing and which batches are at risk right now, from secure login through detection, human review and a closed case, on top of the ColdTrace backend.

1. The product

ColdTrace tells an operations lead, while there is still time to act, that a refrigerated storage unit is failing and which batches are at risk. Today that person learns of a failure after the damage is done.

User. The operations lead at a distribution centre in Johannesburg, Durban or Cape Town. They watch three storage locations, are interrupted constantly, and need one glance to know what needs action now. A reviewer confirms or dismisses each alert, and an auditor checks the evidence afterwards.

Problem. Temperature, cooling and sensor failures are noticed late. Every hour above 8 °C costs product, and false alarms teach people to ignore the system.

What the app does. It shows live status for three locations, raises alerts for three exception types, links each alert to the batches at risk, takes the case through human review to closure, and keeps tamper-evident evidence. This supports SDG 3, Good Health and Well-Being, because spoiled vaccines, medicines and food harm health.

Three exception types the backend detects:

Type	What it means	What the operator does
`TEMP_EXCURSION`	Temperature left the 2 to 8 °C band (door open, power loss)	Check the unit, move or protect batches
`COOLING_FAILURE`	Temperature rising while cooling load is abnormally low (compressor fault)	Call maintenance, move batches
`SENSOR_DROPOUT`	Connectivity collapsed and readings are frozen, so the temperature cannot be trusted	Verify manually, treat as unknown

Success measures for the app:

- From an open alert to a closed case in under 2 minutes and 6 taps or fewer.

- Batches at risk are one tap from any alert.
- Every case carries an evidence hash that shows Verified or Mismatch.
- A false alarm takes one tap to mark, and that feedback is stored for the model evaluation.
- Backend reference points the interface should make visible: the random forest catches 94% of events with a median time to detect of 0.5 hours for excursions and 1.0 hour for cooling failures, at about 0.11 false alerts per location per day. These come from simulated data, so the app must say so.

2. How Newly should use this brief

Build one phase at a time, on mock data first, and treat this brief as the source of truth. Do not build everything in one message.

Platform. Newly builds native iOS and Android apps, so design this as a tablet-first operations console that also works well on a phone. iPad landscape is the largest layout, with a persistent sidebar. If Newly offers a web or desktop target, add a wide layout; if it does not, do not fake one. Use React Native with TypeScript if Newly allows it. Newly's documentation does not state its framework, so confirm it in Ask mode before the first build.

How the pieces fit. The app talks only to the Newly Backend API service. That service holds the secrets and calls the ColdTrace backend on Cloud Run. The ColdTrace backend and its Firestore store are the source of truth for alerts and cases; Newly Backend handles sign-in and acts as a thin proxy.

Standing rules. Paste the block below into a file named `AGENTS.md` at the project root. Newly reads it on every request (limit 20,000 characters).

```
# ColdTrace Operations App: standing rules
Product: operations console for cold-chain storage monitoring. Roles:
operations_lead, reviewer, auditor, admin.
Stack: React Native + TypeScript. Use the framework Newly already uses. Never
add a library without asking.
Data: every screen reads through src/api/client.ts. Two adapters: mock
(default) and live. Screens never call fetch directly.
Secrets: never in the app, never in chat. The live adapter calls only our
Newly Backend API service; the backend holds COLDTRACE_API_URL and
COLDTRACE_INGEST_KEY.
Design: use only tokens from src/theme/tokens.ts. No hard-coded colours. Dark
theme default, light theme supported.
Never rely on colour alone: every status has an icon and a text label.
Time: store UTC, display Africa/Johannesburg (SAST). Units: degrees Celsius
and percent.
Types: use only the fields in src/api/types.ts. Do not invent fields. If data
is missing, show an empty state.
States: every screen has loading (skeleton), empty, error with Retry, and
stale or offline handling.
Roles: enforce on the backend; the UI only hides what the backend already
refuses.
AI text: label any model-written note "AI-generated, review before acting".
It never triggers an action.
Synthetic data: show a "Simulated data" badge in the header whenever the mock
adapter or the simulator is active.
Accessibility: 44 pt touch targets, labels on every control, dynamic type,
contrast AA.
Process: one change per request. Build only the current phase. Run the app
and check it before saying done.
Out of scope: editing detection thresholds, sending real SMS or email,
payments, multi-tenant.
```

3. Design system

The look is a calm control room: dark by default, dense but never cluttered, with colour reserved for status so that a red tile is the only loud thing on screen. Avoid the default template look of generic cards on white; use the tokens below for every screen.

Colour tokens (define once in `src/theme/tokens.ts`):

Token	Dark	Light	Use
`bg`	#0B1220	#F5F7FB	App background

Token	Dark	Light	Use
`surface`	#111A2C	#FFFFFF	Cards, sheets
`raised`	#17233A	#EEF2F9	Selected rows, inputs
`border`	#24324D	#D7DEEA	Dividers, outlines
`text`	#E6EBF5	#0F172A	Primary text
`muted`	#93A1BC	#475569	Secondary text, axes
`accent`	#38BDF8	#0369A1	Links, selected tab, primary button
`ok`	#34D399	#047857	Within 2 to 8 °C, healthy
`warn`	#F5B83D	#B45309	Trending out of band, stale data
`critical`	#F0616D	#B91C1C	Open exception
`unknown`	#94A3B8	#5B6B80	Warming up, no recent reading
`signal`	#A78BFA	#6D28D9	Sensor dropout

Every text and background pair must meet WCAG AA contrast (4.5:1 for body text) in both themes. Status is never colour alone: pair it with an icon and a word.

Exception type marks:

Type	Icon	Label	Colour
`TEMP_EXCURSION`	Thermometer with up arrow	Temperature excursion	`critical`
`COOLING_FAILURE`	Snowflake with slash	Cooling failure	`warn` on `raised` with `critical` border
`SENSOR_DROPOUT`	Signal with slash	Sensor dropout	`signal`

Typography. Inter (or the platform system font) for interface text. JetBrains Mono (or system mono) for readings, IDs and hashes, with tabular figures so numbers do not jump.

Style	Size / weight	Use
Display	28 / 700	Screen titles, big readings
Title	20 / 600	Section headings
Body	16 / 400	Default text

Style	Size / weight	Use
Label	14 / 500	Table headers, chips
Caption	12 / 400	Timestamps, axis labels
Reading	32 / 600 mono	Current temperature on a location card

Layout. 8 pt grid. Corner radius 10 for controls and 14 for cards. One subtle elevation level, no heavy shadows or gradients. Tablet landscape: sidebar of 240 pt plus content with a 12-column grid. Phone: bottom tabs for Command Center, Alerts, Batches and More.

Core components (build once, reuse everywhere):

- `StatusPill`, `TypeBadge`, `KpiTile`, `LocationCard` with sparkline, `AlertRow`, `BatchCard`.
- `ReadingChart`: line chart with the 2 to 8 °C band shaded, event markers, 48-hour default range, down-sampled to 400 points or fewer.
- `Timeline`, `EvidenceTable`, `HashBadge` (Verified, Mismatch, Not checked), `ConfirmSheet`, `FilterChips`, `SegmentedControl`.
- `EmptyState`, `ErrorBanner` with Retry, `Skeleton`, `Toast`, `OfflineBanner`, `SimulatedDataBadge`.

Motion. Short and functional: 150 to 200 ms fades and slides, a gentle pulse on an open critical status, and nothing that loops on a calm screen. Respect the system reduce-motion setting.

4. Roles and navigation

Four roles, each seeing only what they need, with every operational action allowed by role on the backend as well as hidden in the interface.

Capability	Operations lead	Reviewer	Auditor	Admin
View command centre, locations, alerts, batches, detection quality	Yes	Yes	Yes	Yes
Acknowledge an alert, escalate, add a note	Yes	Yes	No	No
Generate the AI incident note	Yes	Yes	No	No
Record a verdict (real, false alarm, needs info)	No	Yes	No	No

Capability	Operations lead	Reviewer	Auditor	Admin
Close a case	Yes	Yes	No	No
Audit and evidence log, hash verification	No	No	Yes	Yes
Users, roles, notification settings	No	No	No	Yes
Demo simulator	No	No	No	Yes

Case lifecycle. `OPEN` (alert raised) to `ACKNOWLEDGED` (a person has taken it) to `CLOSED` (verdict, outcome and root cause recorded). A case can only move forward. Closing needs a verdict, an outcome and a root cause, so no case closes without a human decision.

Screens and navigation. Ten screens. On tablet the sidebar lists the first six plus Detection Quality; Audit and Settings appear only for roles allowed to open them. On phone the bottom tabs are Command Center, Alerts, Batches and More.

#	Screen	Route	Who	Where it lives
1	Login	`/login`	Everyone	Entry, no navigation
2	Command Center	`/`	All roles	Sidebar, first tab
3	Location Detail	`/locations/:id`	All roles	Opened from a location card
4	Alerts Queue	`/alerts`	All roles	Sidebar, second tab
5	Case Review	`/alerts/:id`	All roles (actions by role)	Opened from an alert
6	Batches at Risk	`/batches`	All roles	Sidebar, third tab
7	Detection Quality	`/quality`	All roles	Sidebar, under More
8	Audit and Evidence	`/audit`	Auditor, admin	Sidebar, under More
9	Settings and Access	`/settings`	Admin (profile for all)	Sidebar, under More
10	Demo Simulator	`/demo`	Admin, demo mode only	Sidebar, under More

The core journey the whole app is built around: **Command Center, tap the red location, Case Review, check evidence and batches, record verdict, close.**

5. Screens

Each screen below lists its purpose, content, actions and the states it must handle. Every screen also has loading, empty, error and offline states as set in the standing rules.

1. Login

Purpose: secure entry. **Content:** wordmark "ColdTrace", one-line tagline ("Know before the product spoils"), email and password fields, Sign in button, Forgot password link, and a short notice that access is logged. **Actions:** sign in; on success go to Command Center. **States:** field validation, wrong credentials (generic message, no hint which field), account locked after 5 failed tries, session expired. **Notes:** show the Simulated data badge on this screen when the mock adapter is active, and a demo role picker only in demo mode.

2. Command Center

Purpose: answer "what needs action right now?" in one glance.

- **Header:** location filter, last-updated time, connection state, Simulated data badge.
- **KPI strip:** open exceptions, batches at risk, median time to detect (last 30 days), false alerts per day.
- **Three location cards:** name, status pill, current temperature in the large mono style, 24-hour sparkline with the 2 to 8 °C band, humidity, cooling load, minutes since last reading. A critical card is raised and pulses once.
- **Live alert feed:** newest first, type badge, location, age, one-tap to Case Review.

Actions: open a location, open an alert, acknowledge from the feed. **States:** a location with no reading for over 45 minutes shows Offline in `unknown`; the first 50 readings after start show Warming up.

3. Location Detail

Purpose: see how one unit has behaved and what the detector thinks now.

- Temperature chart (48 hours default, switch to 7 days) with the safe band and event markers.
- Humidity, cooling load and connectivity charts, stacked and aligned on one time axis.
- Detector panel: model label and confidence, rule label, whether they agree.
- Open and recent alerts for this location, and the batches currently stored here.

Actions: change range, tap a marker to open its alert. **States:** frozen readings are drawn as a flat line with a "Sensor frozen" annotation.

4. Alerts Queue

Purpose: work through alerts in priority order. **Content:** table on tablet, cards on phone, with type badge, location, state, age, confidence and at-risk batch count. **Filters:** state, type, location, date range. **Sort:** newest, oldest, highest risk. **Actions:** open, acknowledge (single or selected). **States:** an all-clear empty state that says when the last alert closed.

5. Case Review

Purpose: decide whether the alert is real, act, and close it with provable evidence. This is the most important screen.

- **Header:** type badge, location, state, first seen and raised times, age.
- **Evidence:** the last 12 readings in a table with the chart above it, the model and rule verdicts, and a `HashBadge` showing the evidence hash (shortened, tap to expand) with Verified, Mismatch or Not checked.
- **Batches at risk:** cards with product sensitivity, estimated freshness, spoilage risk and remaining window.
- **AI incident note:** a Generate button. The note (summary, likely cause, batches at risk, three recommended actions) is shown under an "AI-generated, review before acting" label, can be edited, and never triggers anything.
- **Timeline:** raised, acknowledged, notes, verdict, closed, each with person and time.
- **Action bar:** Acknowledge, Add note, Escalate, Record verdict (real, false alarm, needs info), Close case (outcome and root cause required).

States: read-only for the auditor, a closed case is locked, and a hash mismatch shows a red banner and blocks closing until an admin reviews.

6. Batches at Risk

Purpose: show which product to save first. **Content:** list sorted by spoilage risk, then remaining time window. Each row: location, product sensitivity (low, medium, high), estimated freshness, spoilage risk bar, remaining window, delivery urgency, linked alert. **Actions:** open the linked case, filter by location and urgency. **States:** if no exception is open, show "No batches at risk" with the last check time.

7. Detection Quality

Purpose: let the team and reviewers see how far to trust detection. **Content:** a comparison table of threshold only, rules and random forest (event recall, median and 90th percentile time to detect, false alerts per location per day, alert precision), a recall chart by exception type, a confusion matrix of reviewer verdicts against model labels, and a live false-alarm rate from reviewer feedback. A fixed banner reads "Scores come from simulated data and are not real-world accuracy". **Actions:** switch period, export as CSV.

8. Audit and Evidence

Purpose: prove who did what, and that evidence was not altered. **Content:** a chronological log (time, user, role, action, case, evidence hash before and after), filters by user, action and case, and a Verify evidence button that recomputes the hash on the backend and reports the result. **Actions:** verify, export a signed CSV. **States:** read-only; nothing here can be edited or deleted.

9. Settings and Access

Purpose: control who can do what. **Content:** users and roles table, invite and deactivate, session timeout (default 15 minutes idle), notification channels (shown as configured or not, no real sending in this version), an API connection panel showing health and last successful call, and a read-only view of the detection thresholds. **Actions:** admin only; every change is audit-logged.

10. Demo Simulator

Purpose: run the product story for the demo video. **Content:** Start replay (the 14-day sample), speed control, and three buttons that inject a temperature excursion, cooling failure or sensor dropout at a chosen location. A persistent Simulated data badge is shown while it runs. **Actions:** admin only, only when demo mode is on.

6. Data contract

The app uses only the shapes below, through `src/api/client.ts`. Three endpoints exist in the backend today; the rest are for Moorthy to add, so the app runs on a mock adapter until they are live. Paths marked `/api/...` are served by the Newly Backend API service, which forwards to the ColdTrace backend on Cloud Run and attaches the secret key.

Method and path	Status	Purpose	Allowed roles
`GET /healthz`	Live	Backend health, list of locations	Server side only

Method and path	Status	Purpose	Allowed roles
`POST /ingest`	Live, needs `X-API-Key`	Send one reading; returns detection and any new alert	Server side only, never from the app
`POST /alerts/{id}/explain`	Live, needs `X-API-Key`	AI incident note from the NVIDIA NIM model	Operations lead, reviewer
`GET /api/locations`	To build	Status of the three locations	All
`GET /api/locations/{id}/readings?from&to&step`	To build	Readings for charts	All
`GET /api/alerts?state&kind&location&from&to`	To build	Alerts queue	All
`GET /api/alerts/{id}`	To build	One alert with evidence and timeline	All
`POST /api/alerts/{id}/ack`	To build	Acknowledge	Operations lead, reviewer
`POST /api/alerts/{id}/notes`	To build	Add a note	Operations lead, reviewer
`POST /api/alerts/{id}/verdict`	To build	Record verdict	Reviewer
`POST /api/alerts/{id}/close`	To build	Close with outcome and root cause	Operations lead, reviewer
`POST /api/alerts/{id}/verify`	To build	Recompute evidence hash on the server	Auditor, admin
`GET /api/batches/at-risk?location`	To build	Batches at risk	All
`GET /api/metrics/detection`	To build	Evaluation results plus reviewer feedback	All
`GET /api/audit?user&action&case`	To build	Audit log	Auditor, admin
`POST /api/demo/replay`, `POST /api/demo/inject`	To build	Demo mode only	Admin

Times come from the API in UTC (ISO 8601 ending in `Z`) and the app shows them in Africa/Johannesburg. The simulator has no batch IDs or product names yet, so Chantell and Moorthy will add `batch_id` and `product`; until then the mock adapter invents them.

Types (`src/api/types.ts`; the alert and detection shapes match what the backend returns today):

```
type LocationId = "JHB-DC01" | "DBN-DC02" | "CPT-DC03";
type ExceptionType = "TEMP_EXCURSION" | "COOLING_FAILURE" | "SENSOR_DROPOUT";
type CaseState = "OPEN" | "ACKNOWLEDGED" | "CLOSED";
type LocationStatus = "ok" | "warning" | "exception" | "warming_up" |
"offline";
type Role = "operations_lead" | "reviewer" | "auditor" | "admin";

interface Reading { ts: string; T: number; H: number; C: number; S: number }
// deg C, humidity %, cooling load %, connectivity %
interface EvidenceRow extends Reading {
  location_id: LocationId;
  product_sensitivity?: "Low" | "Medium" | "High";
  estimated_freshness_percent?: number;
  spoilage_risk?: number; // 0 to 1
}
interface LocationSummary {
  id: LocationId; city: "Johannesburg" | "Durban" | "Cape Town";
  status: LocationStatus; latest: Reading | null; minutes_since_reading:
number | null;
  open_alerts: number; sparkline: Reading[];
}
interface Alert {
  alert_id: string; location_id: LocationId; kind: ExceptionType;
  first_seen: string; raised_at: string; model_sha256: string;
  evidence: EvidenceRow[]; context: Record<string, unknown>;
  evidence_sha256: string; state: CaseState;
}
interface Detection {
  status: "warming_up" | "ok" | "exception";
  rf_label: ExceptionType | "NONE"; rf_confidence: number; rule_label:
ExceptionType | "NONE";
  alert: Alert | null;
}
interface IncidentNote {
  alert_id: string; evidence_sha256: string;
  explanation: { valid: boolean; model: string; latency_s: number; tokens:
number;
    note: { summary: string; likely_cause: string; batches_at_risk: string;
```

```
            recommended_actions: string[]; needs_human_review: boolean } | {
raw: string } };
}
interface Verdict { verdict: "real" | "false_alarm" | "needs_info"; note:
string; by: string; at: string }
interface Closure {
  outcome: "no_loss" | "partial_loss" | "total_loss" | "false_alarm";
  root_cause: "door_open" | "power_loss" | "compressor_fault" |
"sensor_fault" | "unknown";
  note: string;
}
interface BatchAtRisk {
  batch_id: string; product: string; location_id: LocationId; alert_id:
string;
  sensitivity: "Low" | "Medium" | "High"; estimated_freshness_percent:
number;
  spoilage_risk: number; remaining_window_hr: number; urgency: "Low" |
"Medium" | "High" | "Critical";
}
interface AuditEvent {
  id: string; ts: string; user: string; role: Role; action: string;
  alert_id?: string; evidence_sha256_before?: string; evidence_sha256_after?:
string;
}
```

Starter mock data (put in `src/api/mock/`, same shapes, three locations with one in each state):

```
{
  "locations": [
    {"id": "JHB-DC01", "city": "Johannesburg", "status": "exception",
"minutes_since_reading": 4, "open_alerts": 1,
     "latest": {"ts": "2026-11-10T12:30:00Z", "T": 8.7, "H": 61.2, "C": 31.0,
"S": 96}},
    {"id": "DBN-DC02", "city": "Durban", "status": "ok",
"minutes_since_reading": 2, "open_alerts": 0,
     "latest": {"ts": "2026-11-10T12:32:00Z", "T": 5.4, "H": 66.8, "C": 57.5,
"S": 98}},
    {"id": "CPT-DC03", "city": "Cape Town", "status": "offline",
"minutes_since_reading": 62, "open_alerts": 0,
     "latest": {"ts": "2026-11-10T11:34:00Z", "T": 5.1, "H": 64.0, "C": 54.2,
"S": 38}}
  ],
  "alert": {
    "alert_id": "a1f4c3d2-7b1e-4c1a-9a55-0d3e2b8c6f10", "location_id": "JHB-
DC01", "kind": "COOLING_FAILURE",
```

```
    "first_seen": "2026-11-10T11:30:00Z", "raised_at": "2026-11-
10T12:00:00Z", "state": "OPEN",
    "model_sha256": "2aee3f178ade", "evidence_sha256": "9f2c41d7e0a85b36",
    "evidence": [
      {"location_id": "JHB-DC01", "ts": "2026-11-10T11:30:00Z", "T": 6.1,
"H": 60.4, "C": 41, "S": 97},
      {"location_id": "JHB-DC01", "ts": "2026-11-10T12:00:00Z", "T": 7.8,
"H": 60.9, "C": 33, "S": 96}
    ],
    "context": {"product_sensitivity": "High", "estimated_freshness_percent":
90.2, "spoilage_risk": 0.41}
  }
}
```


7. Security and quality bar

This is the acceptance standard. A phase is not done until its checks pass.

Security

Rule	Requirement
Secrets	No API key, ingest key or NVIDIA key in the app bundle, the repo, chat or logs. Only the Newly Backend API service holds `COLDTRACE_API_URL` and `COLDTRACE_INGEST_KEY`.
Authentication	Email and password through Newly Backend auth. Sessions expire after 15 minutes idle and 8 hours absolute; the token lives in secure device storage, never in plain storage.
Authorization	Roles are enforced on the server for every endpoint in section 6. The app hides controls a role cannot use, but hiding is cosmetic. Test each role against each endpoint.
Transport	HTTPS only. No cleartext fallback.
Input	Validate and length-limit every field (notes 1,000 characters, no HTML). Reject unknown fields.
Audit	Every acknowledge, note, verdict, close, verify, login, logout and failed login writes an `AuditEvent` with user, role and time. Audit rows cannot be edited or deleted from the app.

Rule	Requirement
Evidence integrity	The Audit and Evidence screen shows the stored `evidence_sha256` next to a hash recomputed on the server. A mismatch shows a clear red tamper warning.
AI text	Every model-written note carries "AI-generated, review before acting". It never closes, acknowledges or changes anything.
Privacy	Show only what the screen needs. No personal data beyond name, email and role.

Quality

Area	Requirement
States	Each screen has loading skeleton, empty, error with Retry, and stale or offline banner. Stale means the newest reading is more than 45 minutes old.
Accessibility	44 pt targets, labels on every control, dynamic type up to 130 percent, contrast AA, status never by colour alone, screen reader order checked.
Performance	Command Center interactive in under 2 seconds on the mock adapter. Charts cap at 500 points per series by downsampling. Lists virtualise.
Consistency	Only tokens from `src/theme/tokens.ts`; shared components for status chip, stat tile, chart and case card.
Honesty	"Simulated data" badge whenever the mock adapter or simulator is active. Detection Quality always shows that the figures come from synthetic data.
Tests	Unit tests for the data adapters and role gating; a smoke test that opens every screen on the mock adapter.
Done means	The app was run, each screen opened, and the phase's acceptance checks passed. Newly reports what it checked and what it could not.

8. Build order

One phase per prompt, in this order, matching the prompting manual. Do not start a phase before the previous one passes its checks.

Phase	Build	Acceptance checks
0	Standing rules and plan. Put the `AGENTS.md` block from "How Newly should use this brief" in place. In Plan mode, produce the file structure and screen list, no code.	`AGENTS.md` saved and under 20,000 characters; plan lists all ten screens and the folders `src/theme`, `src/api`, `src/api/mock`, `src/screens`, `src/components`.
1	Foundations and login. Tokens, dark and light themes, fonts, shared components, navigation shell, Login with role-based routing, session timeout.	Both themes render; all four roles land on the right home screen; wrong password shows a clear error; idle timeout signs out.
2	Command Center and Location Detail on the mock adapter.	Three locations in three different states; status shown by icon, text and colour; chart shows the 2 to 8 C band; stale banner appears when mock time is advanced.
3	Alerts Queue and Case Review. Case lifecycle OPEN, ACKNOWLEDGED, CLOSED, notes, verdict, closure, AI note panel with label.	Filters work; a case moves through all three states; a role without permission cannot act; AI note shows the label and cannot change state.
4	Batches at Risk, Detection Quality, Audit and Evidence.	Batches sorted by urgency; Detection Quality shows the synthetic-data notice; verify shows match and mismatch cases; auditor can read but not act.
5	Settings and Access, Demo Simulator.	Admin manages users and roles; non-admins cannot open these screens; the simulator injects each of the three exception types and the alert appears in the queue.
6	Wire the real API through the Newly Backend proxy. Switch the adapter to live behind a setting; no screen changes.	Every screen works against the live adapter; no secret appears in the app bundle or network traces from the app; errors from the backend show the Retry state.

Phase	Build	Acceptance checks
7	Hardening. Accessibility pass, performance pass, empty and error states, tests, README.	Section 7 checks all pass; smoke test opens all ten screens; reviewer signs off on the walkthrough of one full incident from detection to closure.

If a phase fails a check, fix it in a separate prompt that names the failing check. Do not combine fixes with new features.

9. Delivery

GitHub. Work on a team branch, never straight on `main`. Open a pull request to `main` and leave the merge to the team lead after review. Commit messages say what changed and which phase it belongs to. No keys, no `.env` files and no exported user data in the repo.

Newly and GitHub. The Newly documentation we could read is inconsistent about whether GitHub sync works in both directions. Before relying on it, test it once with a throwaway change. If sync does not work, use the fallback: `newly pull <project-id>` to bring the project files down, then commit them to the team repo by hand.

README. The repo README must cover: what the app is, the ten screens, how to run it on the mock adapter, how to switch to the live adapter, the environment variables held by the Newly Backend service (names only, never values), the roles and how to add a user, and the known limits.

Known limits to state plainly

- The detection data is synthetic; the figures show the pipeline works and ranks methods, not real-world accuracy.
- Newly builds native iOS and Android apps. Web output and the exact framework were not confirmed; the brief assumes React Native with TypeScript and a tablet-first layout.
- Only `/healthz`, `/ingest` and `/alerts/{id}/explain` exist in the backend today. The other endpoints in section 6 must be built before phase 6.
- Alert state is held in memory in the backend until the Firestore integration is finished.
- The AI incident note depends on the NVIDIA model run passing validation; detection does not depend on it.

Out of scope: editing detection thresholds, sending real SMS or email, payments, multi-tenant use.